

Project Practical Parallel Programming: Parallelizing Knapsack

Short Problem Description

The 0/1 knapsack problem is a classic problem in combinatorial optimization. The problem is the following: A knapsack, which can support a certain maximum weight, can be filled with objects. The objects to be included are chosen from a set of objects, each with a weight and a value. The objective is to choose the subset of objects for which the combined value is the largest without the combined weight exceeding the maximum weight the knapsack can support. The knapsack is of course more of a metaphorical device to explain the problem, and one can easily think of numerous real-life applications of the problem set-up in the context of resource allocation. The 0/1 denotes that each object can only be included once. Other variants of the problem where multiple copies of the same object can be included, exist as well.

CPU Info

Using the CPU-Z tool, I get the following information on my CPU. The specification of my CPU is Intel Core i9-11900H with a clock frequency of 2.50 GHz. The CPU consists of 8 physical cores, which amount to 16 logical processors. So through hyperthreading the 8 physical cores of my CPU appear as 16 logical processors.

CPU Mainboard Memory SPD Graphics Bench About						
Processor						
Name	Intel Core i9 11900H					
Code Name	Tiger Lake	Max TDP	45.0 W			
Package	Socket 1787 FCBGA					
Technology	10 nm	Core VID	0.805 V			
Specification	11th Gen Intel® Core™ i9-11900H @ 2.50GHz					
Family	6	Model	D	Stepping	1	
Ext. Family	6	Ext. Model	8D	Revision	R0	
Instructions	MMX, SSE (1, 2, 3, 4.1, 4.2), SSSE3, EM64T, VT-x, AES, AVX, AVX2, AVX512, FMA3, SHA					
Cores (Core #0)						
Core Speed	2493.90 MHz					
Multiplier	x25.0 (8.0 - 48.0)					
Bus Speed	99.76 MHz					
Rated FSB						
Cache						
L1 Data	8 x 48 KBytes		12-way			
L1 Inst.	8 x 32 KBytes		8-way			
Level 2	8 x 1.25 MBytes		20-way			
Level 3	24 MBytes		12-way			
Selection						
Selection	Socket #1		Cores	8	Threads	16

Different Developed Versions

A common approach to solving combinatorial optimization problems is the branch-and-bound approach. In branch-and-bound, the set of possible solutions to the optimization problem is explored through a search tree. Branch-and-bound starts from the root node, which consists of the whole solution set. At each node, the algorithm then makes a decision on one of the variables, effectively creating child nodes based on this decision. For example in the case of the 0/1 knapsack problem, such a decision could be to include a particular object. One child node then represents the subset of the solution set where the object is included, and the other child node represents the subset of the solution set where the object is not included. By making such decisions at each level of depth, the algorithm recursively explores the whole search tree.

To make the exploration of the search tree more efficient, a local upper bound (in the case of a maximization problem) is computed at each node. If the local upper bound is lower than the

value of the best solution the algorithm has found thus far, the node is not further explored, effectively pruning that branch. This pruning based on a bounding function makes it very beneficial to find good solutions early on in the search tree. If a good solution is found early in the search tree, the algorithm is able to prune a large part of the search tree which saves a lot of time. Hence, it is beneficial to steer the search tree to explore promising solutions first. To determine what constitutes a promising solution, some heuristic must be used.

All three versions I have developed, follow this branch-and-bound approach. They only differ in how the exploration of the search tree is distributed across threads.

Sequential version

The sequential version doesn't distribute the exploration of the search tree. Hence, it goes through the search tree sequentially, exploring one branch at a time.

The branching and bounding mechanisms are as follows: As a first step, the algorithm sorts the objects based on the value over weight ratio. Objects with a high value but low weight intuitively are interesting to include in the solution. At each node, the algorithm goes through the sorted objects and includes each object in a provisional solution as long as the inclusion of the next object won't make the provisional solution exceed the maximum weight. If an object is encountered which would push the provisional weight over the maximum, we branch on this object: one branch will explore the subset of the solution set with this object included, the other branch will explore the subset with this object excluded. From this provisional solution, the local upper bound of the branch can also be calculated by adding the fractional portion of the value of this object to the provisional value (fraction determined by the weight of the object over the provisional remaining weight). If this local upper bound is lower than the already found best value, the child nodes will not provide better solutions and hence are pruned.

Before branching, the lower feasible bound is also calculated for both child nodes. The child node with the highest feasible bound is pursued first. The algorithm recursively goes deeper until a node is pruned or until a leaf node is reached, i.e. a node for which a decision has been made on all variables, which represents a complete solution. The described branch-and-bound approach also forms the basis for the multithreaded versions.

Besides a few adjustments, the code for the sequential version is code I had developed for an assignment of the Combinatorial Optimization course of Erik Demeulemeester at KU Leuven.

Multithreaded Version

As mentioned above, the multithreaded version follows exactly the same branch-and-bound approach, but the exploration of the search tree is distributed across a number of threads. More precisely, as long as the maximum number of threads isn't reached, each time a branching decision is made, one of the branches is assigned to a newly created thread. So when a node makes a branching decision on an object, it checks if the maximum number of threads isn't reached, and if it hasn't, it assigns the exploration of the first branch to a newly created thread. The second branch is further explored in the current thread. Since the branching decision is always made within a thread, the threads create each other in a sort of distributed fashion.

Multithreaded Version V2: re-use of threads

After testing the first multithreaded version, some modest speed-ups could already be noticed. However, my intuition told me it should be possible to still do better. In branch-and-bound, very often a branch is explored only for a few nodes before being pruned by the bounding mechanism. Hence, in the first multithreaded version, a lot of the threads we create will only explore one or two nodes before being finished. This means that the work we save by distributing the exploration isn't necessarily enough to compensate for the work of a new thread creation. Therefore, in the second multithreaded version the threads are re-used. Once a thread has finished exploring a branch, it puts its status back to inactive and goes in waiting mode. In this way other threads know they can assign new work to this inactive thread. So, in this version the threads are created centrally by the main function, but they wake each other up in a distributed fashion. Each time a branching decision is made, the branching thread checks if there is an inactive thread, and if there is, the first branch is assigned to the inactive thread and the inactive thread is woken up. Before the activated thread can start its exploration, the activating thread first has to initialize a starting state for the activated thread. This is still a considerable overhead whenever we assign work to a thread.

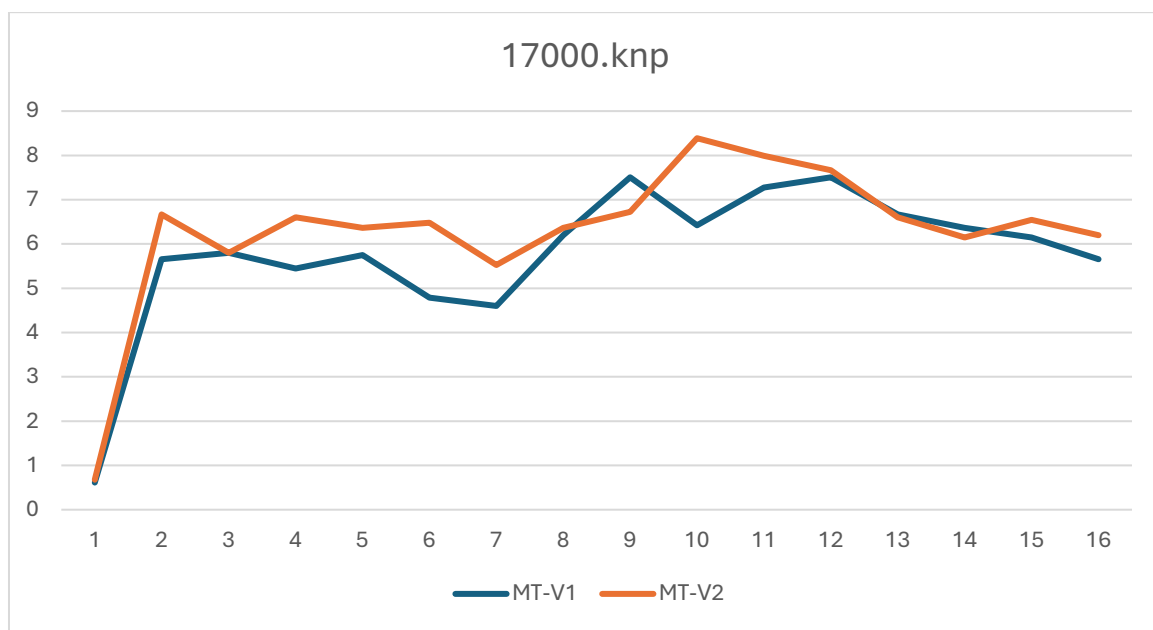
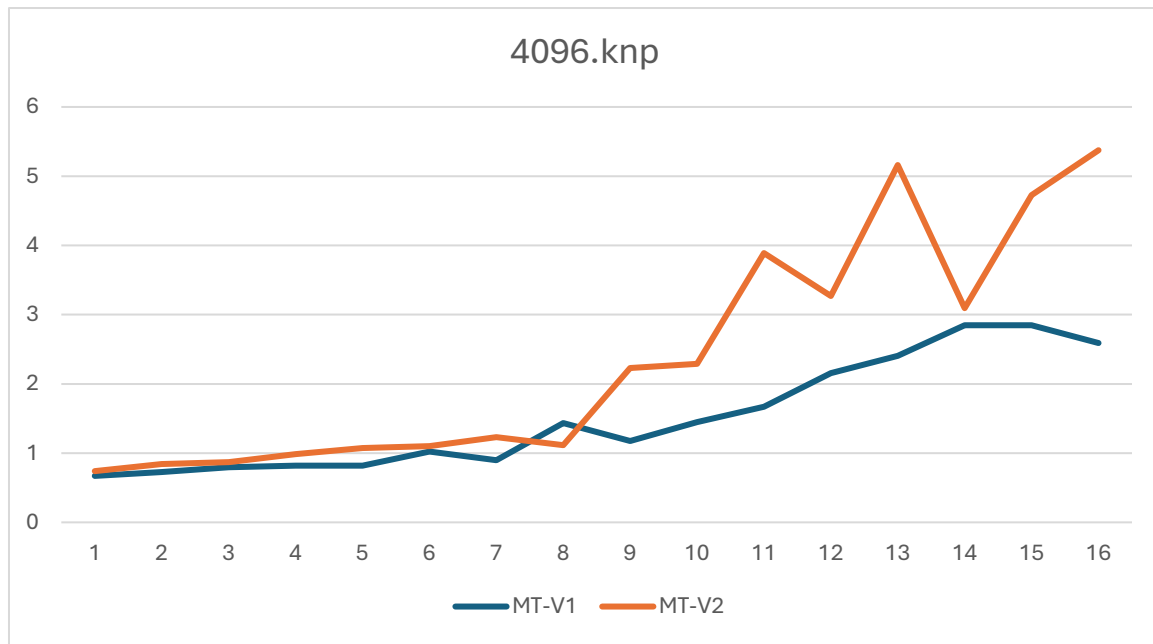
Performance Analysis

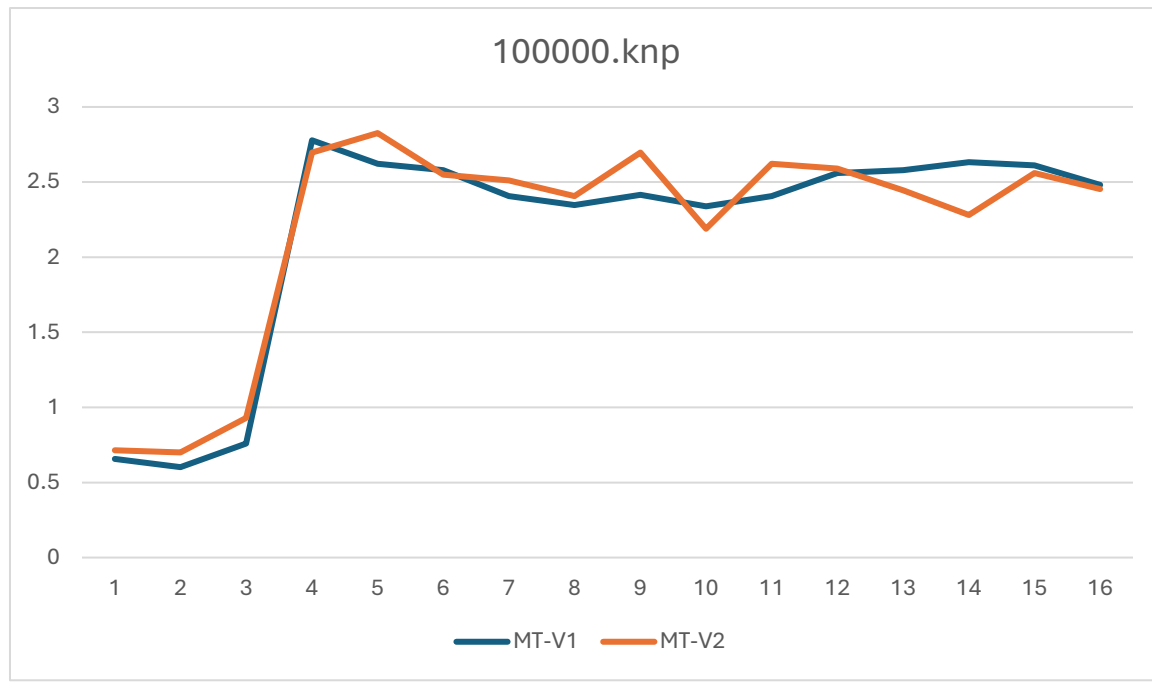
In order to test the performance of the different versions, I tested the versions on a few knapsack instances. As part of the assignment of the Combinatorial Optimization course at KU Leuven, professor Erik Demeulemeester provided us with a number of knapsack instances: '8.knp', '10.knp', '20.knp', '30.knp', '1536.knp' and '4096.knp'. All of these, except for the '4096.knp', are easy instances which the sequential version solves in 0.00-0.02 seconds. For these instances, the overhead of the thread creation and management leads to these easy instances being solved slightly slower, in between 0.00-0.03 seconds. In other words, for such small and easy instances which the sequential solver solves very quickly anyway, switching to a multithreaded version is not a good idea. It is also good to note that the 'hardness' of a knapsack instance is not only defined by the problem size, i.e. the number of objects to choose from. Whether an instance is hard to solve, also depends on the specific structure of the instance. For example, instances with a high correlation between the weight and value of an object are harder to solve (Pisinger, 2004). There are several research papers which discuss what constitutes a hard knapsack instance.

It is for these hard knapsack instances that the multithreaded version can provide a tangible benefit. Hence, I will concentrate the performance analysis on three such 'hard' instances. One of these three is the '4096.knp'. In order to have two more large hard instances, I consulted ChatGPT to generate a simple Python script which creates such hard instances, leading to the inclusion of the '17000.knp' and '100000.knp' instances. The graphs of the speed-ups for the multithreaded versions on these three instances are shown in the next subsection.

Speed-ups

Below are the graphs of the speed-ups obtained by the multithreaded versions in relation to the number of threads used. The results for a range of 1 to 16 threads are shown. In order to calculate these speed-ups, all versions were run 5 times and the computation time averaged to increase the reliability of the presented results.





The exact behavior of the speed-ups depends on the specific problem instance, but some commonalities can be discerned. For both the '4096.knp' and the '100000.knp' instance, the multithreaded versions need to use a number of threads before they outperform the sequential version. For the '4096.knp' instance, this happens from 6 threads onwards. For the '100000.knp' instance, this happens from 4 threads onwards. For the '17000.knp' instance the multithreaded versions strongly outperform the sequential version ($\sim x6$) already from 2 threads onwards. For fewer threads than this, the overhead associated with thread creation and thread management dominates. In general, we also observe that the second version of the multithreaded approach outperforms the first version, reaching speed-ups of ~ 5.4 for the '4096.knp' instance, speed-ups of ~ 8.4 for the '17000.knp' instance and speed-ups of ~ 2.8 for the '100000.knp' performance. The increased performance of the second multithreaded version is most pronounced for the '4096.knp' instance.

In general, we also see that speed-ups plateau and don't significantly increase any further past 12-13 threads. Results on performance with more than 16 threads for both multithreaded versions are not shown in the graph, but I can report from experimentation that increasing the thread count beyond 16 threads doesn't offer increased performance for any of the instances. Instead, performance past this point keeps slowly decreasing. This can be explained by the fact that an increased thread count also increases the contention on shared variables across the different threads.

References

PISINGER, D. (2004). Where are the hard knapsack problems? Computers & Operations Research. [https://doi.org/10.1016/s0305-0548\(04\)00036-x](https://doi.org/10.1016/s0305-0548(04)00036-x)